MEET THE HOUNDS

PYTHIA

AI Inventory

Pythia Hound · Category: Discovery

"Reveal the AI hiding in your estate."

The oracle of Delphi — Pythia sees what others can't: the shadow AI already running in your environment.

THE PROBLEM

AI and ML tools spread faster than any inventory can track — Cursor, Claude Code, Ollama, Jupyter, vector databases — and each one can ship sensitive data to third-party APIs. Vulnerability scanners weren't built to see "AI risk," so shadow AI accumulates invisibly until an auditor, or an incident, finds it first.

WHAT IT DOES

Pythia discovers shadow AI by content, classifies it by role, and governs where its data flows. It sweeps five evidence sources, classifies each find — Model Serving, Vector/RAG, GPU/Training, MLOps, LLM client, AI Dev Tool — flags **exposed endpoints** (often unauthenticated), correlates **KEV** exploitability, and separates **sanctioned from shadow data egress** via an editable allowlist.

KEY CAPABILITIES

- **5-source discovery** — sweeps the AI plugin family, software inventory, CPEs, and plugin output into one AI asset list that catches what any single source misses.
- **Role classification + per-role ACR** — every find typed as Model Serving, Vector/RAG, GPU/Training, MLOps, LLM client, or AI Dev Tool, with matching criticality — a GPU training box shouldn't score like a chat client.
- **Exposed-endpoint and unauthenticated detection** — flags the AI services reachable right now, often with no authentication — the findings that turn a governance report into incident prevention.
- **Data-egress governance** — detects egress capability and configured destinations, split sanctioned vs. shadow by an editable allowlist — "where can our data go?" finally has an answer.
- **"Why?" evidence inspector + FP suppression** — every detection shows its evidence, and false positives can be suppressed — the inventory stays trusted instead of argued with.
- **MITRE ATLAS mapping + gated "AI" tags** — findings carry adversary-technique context from the live ATLAS catalog and become human-approved, routable tags.

HOW IT WORKS

Pythia runs against the local navi.db built from your Tenable data: the **vulns** AI plugin family and output, plus the **software** and **cpes** inventories, with a browser fetch of the MITRE ATLAS catalog for technique mapping. All writes are proposed, human-approved, and logged.

WHY IT'S DIFFERENT

- **Finds AI by evidence, not a memorized list** — it catches Cursor, Claude Code, OpenAI clients, and Ollama as CPEs even when no "AI plugin" fired.
- **Evidence-first with FP control** — every detection carries its "why," and you can suppress false positives instead of arguing with them.
- **Honest about egress:** Pythia reports egress **capability and configured destinations** — not proven exfiltration — and says so plainly.

PROOF POINTS

In a reference environment of 268 assets, Pythia surfaced:

- **29 assets** via the AI plugin family.
- **15 additional AI-native apps** found only through CPE evidence — invisible to plugin-only detection.

Illustrative results from a demo lab — not a guarantee. Egress findings are capability plus configured destinations, not proven exfiltration; blind or uncredentialed hosts are called out, not hidden.

WORKS BETTER WITH

Pythia feeds **Fenrir** — an exposed, unauthenticated AI endpoint is an attack-path entry — and **Anubis**, which turns per-role ACR suggestions into calibrated criticality. It shares the KEV signal with **Laelaps**.

WHO IT'S FOR

AI governance and risk teams who need a defensible inventory; security architects deciding what to sanction; CISOs answering "how much AI do we run, and where does its data go?"

CALL TO ACTION

Ask Pythia what AI is running in your estate — the oracle already knows.